Instituto Tecnológico de Estudios Superiores Occidente

Redes para Sistemas Embebidos

Práctica: Thread

Bajo conducción de:

Estephania Martínez García

Alumnos:

Pablo Emmanuel Valencia Hernández

Roberto Nahum Gutiérrez Gallardo

INTRODUCTION

This practice uses, for both routers, the SDK example called "router_eligible_device", which allows the KW41Z board to connect to a mesh network and, through the Thread protocol, read its temperature sensor, control LEDs, and communicate with other boards. Additionally, the Router 1 modifications for Part 3 regarding the accelerometer used the logic from the SDK's bubble example.

THEORETICAL FRAMEWORK — THREAD

Thread is a wireless network protocol designed by NXP for IoT. It runs on top of IEEE 802.15.4 and uses IPv6 to address devices. Its main characteristic is that it is a mesh-type network, where devices assist each other in packet routing.

Thread network roles:

- **Leader:** Manages the network, assigns IDs, unique per partition.
- **Router:** Routes packets between devices, can become Leader.
- **REED:** Router Eligible End Device, can become a Router.
- **End Device:** Only communicates with its parent node, does not route.

The example used is a Router Eligible Device.

CoAP

Constrained Application Protocol is a communication protocol designed specifically for resource-constrained devices such as microcontrollers, sensors, and IoT nodes. It was defined by the IETF in RFC 7252. It works as a simplified HTTP but designed for small networks with limited memory and low bandwidth.

Devices & Types.

1. Leader The Leader is the administrator of the Thread network. Only one is active at a time per network partition. It is responsible for assigning IDs to routers, coordinating commissioners, and making decisions about network parameters. If the Leader fails, another Router automatically takes its place, which is why Thread has no Single Point of Failure. In this practice, Router 1 took the Leader role when creating the network.

2. Active Router Forms the backbone of the mesh network. It routes traffic between devices and can communicate directly with other routers. A Thread network can have up to 31 Active Routers. They are always on and generally powered by continuous current. In this practice, Router 2 operated as an Active Router upon joining the network.

3. Router Eligible End Device (REED) A device that has the capability to become an Active Router but has not yet done so. When it joins the network through an existing router and the network has sufficient connectivity, it operates as a REED. If the network needs more routers (when the total number of Active Routers is below 16), the REED requests the Leader to become an Active Router. It is always on and powered by continuous current.

4. End Device Has no routing capabilities. It communicates only through its parent Active Router and cannot become a router. It may be powered by continuous current but can be turned off periodically, or it may have a high-capacity battery.

5. Sleepy End Device (SED) An End Device with its radio mostly off to maximize power savings. It uses data polling to check whether its parent router has messages stored for it. It cannot become a router and is intended for small non-rechargeable batteries such as a coin cell.

IPv6 addresses.

The addresses are not hardcoded; they are obtained at runtime when the board connects, in the following case.

```
        break;

    case gThrEv_GeneralInd_Connected_c:
        App_UpdateStateLeds(gDeviceState_NwkConnected_c);
        /* Set application CoAP destination to all nodes on connected network */
        THR_GetIP6Addr(mThrInstanceId, gAllThreadNodes_c, &gCoapDestAddress, NULL);
        APP_SetMode(mThrInstanceId, gDeviceMode_Application_c);
        mFirstPushButtonPressed  = FALSE;
        /* Synchronize server data */
        THR_BrPrefixAttrSync(mThrInstanceId);
        /* Enable LED for 80215.4 tx activity */
        gEnable802154TxLed = TRUE;
```

Ilustración 1: router_eligible_device_app.c

PanID.

```
/*! The PAN identifier.
    If this value is 0xFFFF a random PAN ID will be generated on network creation */
#ifndef THR_PAN_ID
    #define THR_PAN_ID                                  THR_ALL_FFs16
#endif
```

Figure 1: app_thread_config.h

```
53
54 /*! Max unsigned 16bit integers value */
55 #define THR_ALL_FFs16                               0x3333
56
```

Figure 2: network_utils.h

Channel.

The channel assigned to our team was 26. Although it can be set through the terminal, for convenience it was changed to 26 by default.

```
115 /*********************************/
116 /* 802.15.4 default configuration */
117 /*********************************/
118 /*! The channel mask used when a network scanning is performed (energy scan, active scan or both
119     0x07FFF800 sets all 2.4GHz 16 channels are used (from channel id 11 to 26).
120     This channel mask is used during network creation to select the best channel or during networ
121     to find the available networks on that channels.
122     To set the scan mask to a single channel, set the mask to 0x00000001 shifted with the channel
123     e.g.: to set mask for channel 25, set THR_SCANCHANNEL_MASK to: (0x00000001 << 25) */
124 #ifndef THR_SCANCHANNEL_MASK
125     #define THR_SCANCHANNEL_MASK                    (0x00000001 << 26)
126 #endif
127
```

Ilustración 2: app_thread_config.h

Network name.

The network name was not modified.

```
199 #endif
200
201 /*! The default network name */
202 #ifndef THR_NETWORK_NAME
203     #define THR_NETWORK_NAME                        {14, "Kinetis_Thread"}
204 #endif
205
```

Ilustración 3: app_thread_config.h

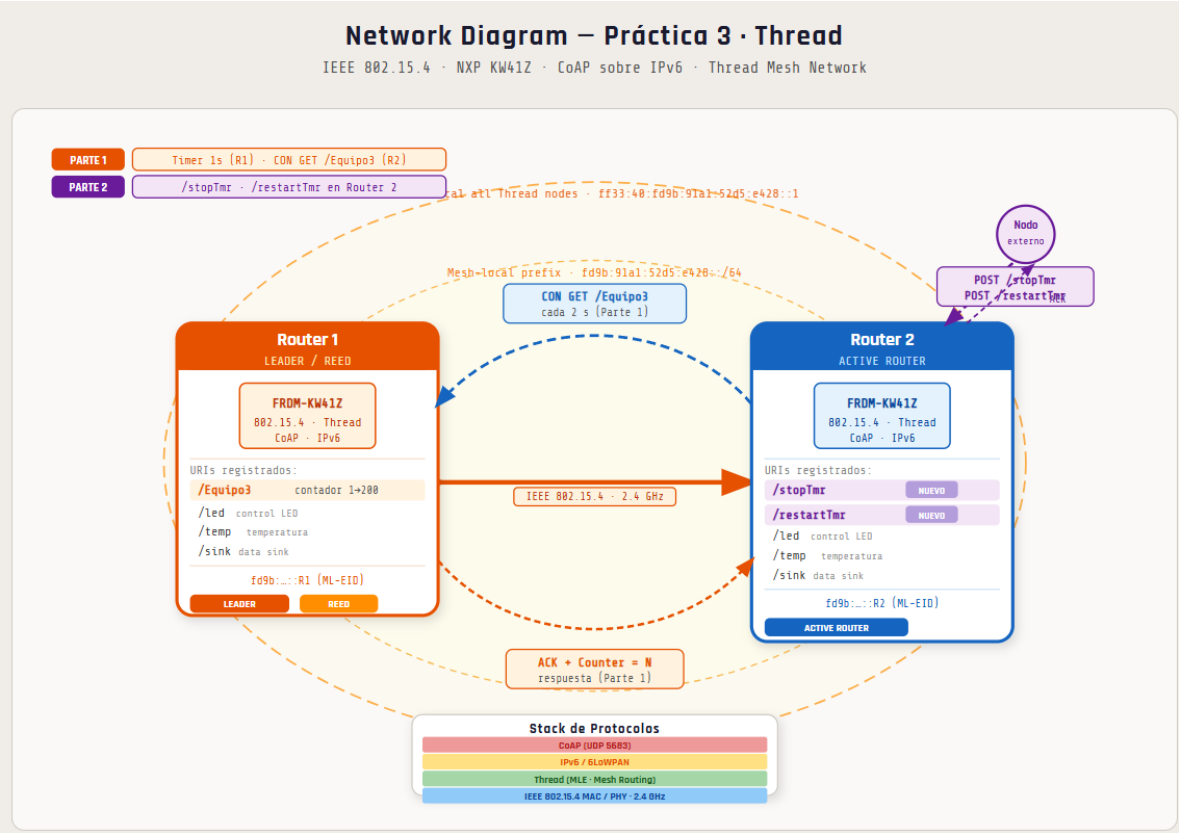
Timer on Router_1.

```
68      #define shell_refresh()
69      #define shell_printf(a,...)
70  #endif
71
72  #define gThrDefaultInstanceId_c                 0
73  #if APP_AUTOSTART
74  #define gAppFactoryResetTimeoutMin_c            10000
75  #define gAppFactoryResetTimeoutMax_c            20000
76  #endif
77  #define gAppRestoreLeaderLedTimeout_c           60      /* seconds */
78
79  #define gAppJoinTimeout_c                       800     /* miliseconds */
80
```

Ilustración 4: router_eligible_device_app.c

Timer on Router_2 to request counter.

Network diagram.



Sequence diagram.

Router_1:

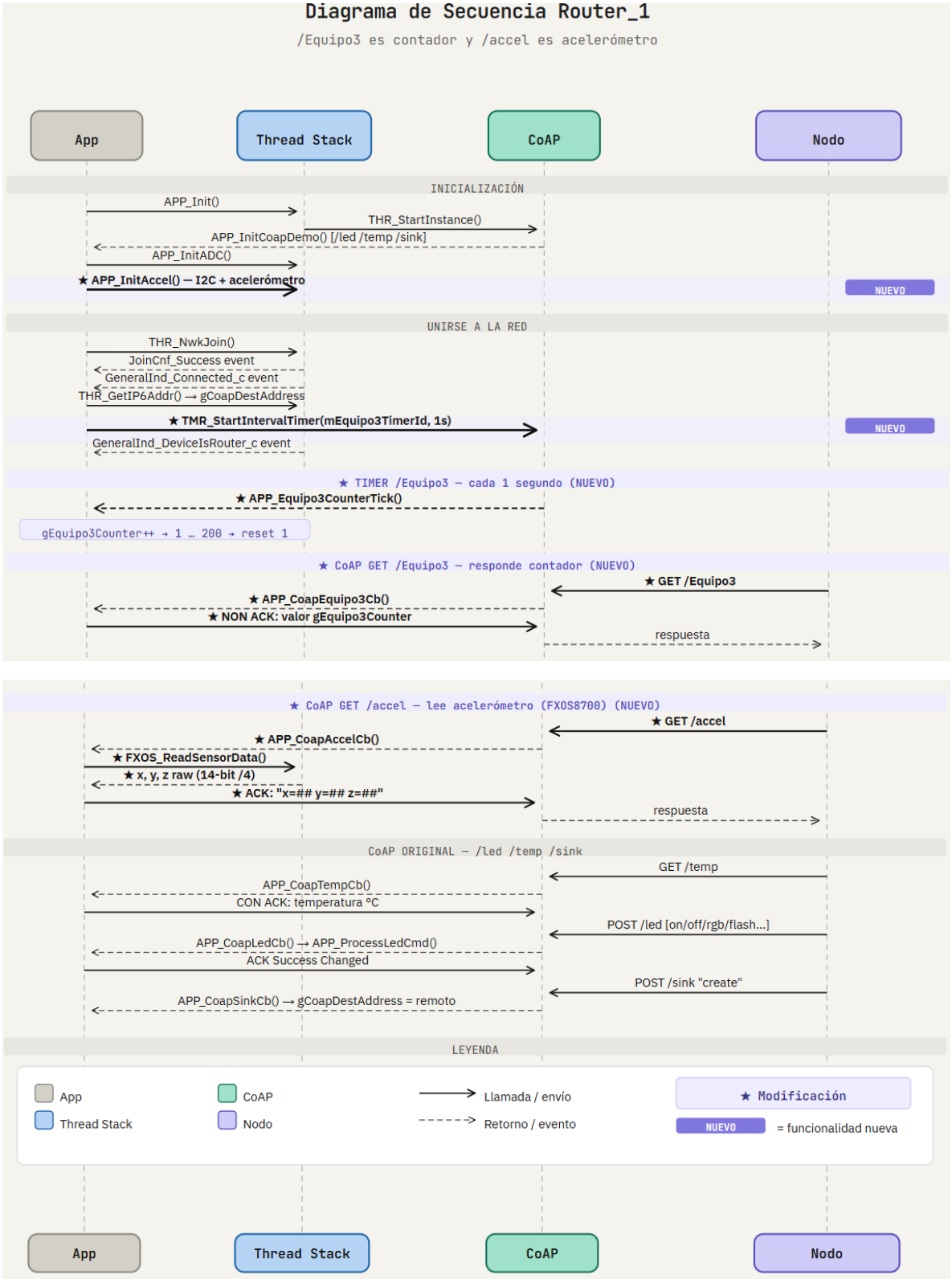
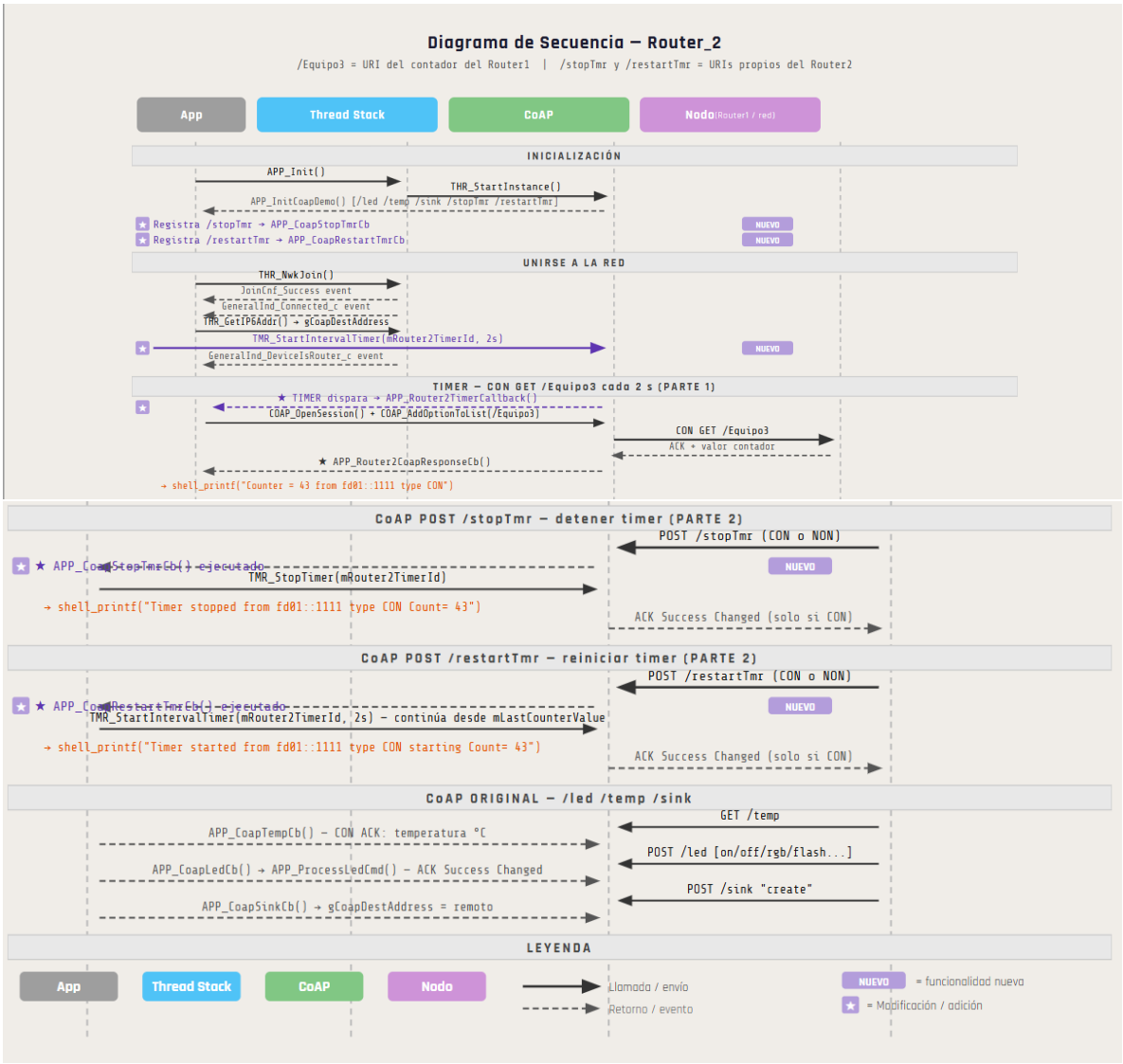


Diagrama de Secuencia Router_1
/Equipo3 es contador y /accel es acelerómetro
App
Thread Stack
CoAP
Nodo
INICIALIZACIÓN
APP_Init()
THR_StartInstance()
APP_InitCoapDemo() [/led /temp /sink]
APP_InitADC()
★ APP_InitAccel() — I2C + acelerómetro
NUEVO
UNIRSE A LA RED
THR_NwkJoin()
JoinCnf_Success event
GeneralInd_Connected_c event
THR_GetIP6Addr() → gCoapDestAddress
★ TMR_StartIntervalTimer(mEquipo3TimerId, 1s)
NUEVO
GeneralInd_DeviceIsRouter_c event
★ TIMER /Equipo3 — cada 1 segundo (NUEVO)
★ APP_Equipo3CounterTick()
gEquipo3Counter++ → 1 … 200 → reset 1
★ CoAP GET /Equipo3 — responde contador (NUEVO)
★ GET /Equipo3
★ APP_CoapEquipo3Cb()
★ NON ACK: valor gEquipo3Counter
respuesta
★ CoAP GET /accel — lee acelerómetro (FXOS8700) (NUEVO)
★ GET /accel
★ APP_CoapAccelCb()
★ FXOS_ReadSensorData()
★ x, y, z raw (14-bit /4)
★ ACK: "x=## y=## z=##"
respuesta
CoAP ORIGINAL — /led /temp /sink
GET /temp
APP_CoapTempCb()
CON ACK: temperatura °C
POST /led [on/off/rgb/flash...]
APP_CoapLedCb() → APP_ProcessLedCmd()
ACK Success Changed
POST /sink "create"
APP_CoapSinkCb() → gCoapDestAddress = remoto
LEYENDA
App
Thread Stack
CoAP
Nodo
Llamada / envío
Retorno / evento
★ Modificación
NUEVO
= funcionalidad nueva
App
Thread Stack
CoAP
Nodo

Router_2:



Is Thread useful? Why?

Thread is useful because it allows devices such as multiple KW41Z boards to automatically form a mesh network where roles are self-assigned and the network can recover without manual intervention.

What went wrong in the process?

There was a misunderstanding of the practice statement. The timer we had to stop was the router's own timer, not the counter that was implemented. Because of this, it was not possible to send messages through the terminal since it kept printing messages without stopping.

We were using `mTeamCounter`, which is a local variable in the Router 1 file. Router 2 has no access to it because they are separate projects running on different devices.

Since we used it directly in Router 2, we received an undeclared identifier error. The solution was to store the last received value in a local variable of Router 2, `mLastCounterValue`, and update it inside `APP_Router2CoapResponseCb`.

Was it easy to implement?

This was a moderately complex practice to implement, as it required adding URIs, correctly understanding the CoAP flow, knowing how to register callbacks, handling CON and NON message types, and implementing accelerometer reading. It was not difficult in the sense of complex algorithms, but it was challenging in the sense of properly understanding the system architecture before being able to modify it correctly.

Modified code on Router_1.

In addition to the PAN ID and channel modifications, the following changes were made to Router 1.

INCLUDES

```
57 #include "fsl_fxos.h"
58 #include "fsl_i2c.h"
```

Ilustración 5: includes para acelerómetro, también se agregó stdio.h para snsprintf

MACROS Y CONSTANTES

```
87
88 #define APP_EQUIPO3_URI_PATH                    "/Equipo3" /*URI*/
89 #define APP_ACCEL_URI_PATH                      "/accel"
90
91 /* Counter limits */
92 #define APP_EQUIPO3_COUNTER_MIN_c               1
93 #define APP_EQUIPO3_COUNTER_MAX_c               200
94 #define APP_EQUIPO3_TIMER_PERIOD_c              1000    /* milliseconds */
95
```

Ilustración 6: URIs, Constantes para contador y periodo del contador.

PROTOTIPOS DE FUNCIONES NUEVAS

```
144 /* Equipo3 prototypes */
145 static void APP_CoapEquipo3Cb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataL
146
147 static void APP_Equipo3TimerCallback(void *param);
148 static void APP_Equipo3CounterTick(uint8_t *pParam);
149
150 static void APP_CoapAccelCb(coapSessionStatus_t sessionStatus, uint8_t *pData, coapSession_t *pSession, uint32_t dataLen
151 static void APP_InitAccel(uint8_t *param);
152
```

VARIABLES GLOBALES

```
162
163 const coapUriPath_t gAPP_EQUIPO3_URI_PATH = {SizeOfString(APP_EQUIPO3_URI_PATH), (uint8_t *)APP_EQUIPO3_URI_PATH};
164
```

```
186
187 /* Counter and Timer*/
188 uint16_t gEquipo3Counter = APP_EQUIPO3_COUNTER_MIN_c;
189 tmrTimerID_t mEquipo3TimerId = gTmrInvalidTimerID_c;
190
191 const coapUriPath_t gAPP_ACCEL_URI_PATH   = {SizeOfString(APP_ACCEL_URI_PATH),   (uint8_t *)APP_ACCEL_URI_PATH};
192
193 /* Accelerometer handle and I2C master handle */
194 fxos_handle_t gFxosHandle;
195 i2c_master_handle_t gAccelMasterHandle;
196 /* FXOS device candidate addresses */
197 static const uint8_t mAccelAddress[] = {0x1CU, 0x1DU, 0x1EU, 0x1FU};
```

REGISTRO DE URIS EN APP_InitCoapDemo

```
537 static void APP_InitCoapDemo
538 (
539     void
540 )
541 {
542     coapRegCbParams_t cbParams[] =  {{APP_CoapLedCb,  (coapUriPath_t *)&gAPP_LED_URI_PATH},
543                                      {APP_CoapTempCb, (coapUriPath_t *)&gAPP_TEMP_URI_PATH},
544 #if LARGE_NETWORK
545                                      {APP_CoapResetToFactoryDefaultsCb, (coapUriPath_t *)&gAPP_RESET_URI_PATH},
546 #endif
547                                      {APP_CoapSinkCb, (coapUriPath_t *)&gAPP_SINK_URI_PATH},
548                                      {APP_CoapEquipo3Cb, (coapUriPath_t *)&gAPP_EQUIPO3_URI_PATH},
549                                      {APP_CoapAccelCb,   (coapUriPath_t *)&gAPP_ACCEL_URI_PATH}};
550     /* Register Services in COAP */
551     sockaddrStorage_t coapParams = {0};
552
```

ARRANQUE DEL TIMER Y ACELERÓMETRO

```
356
357         case gThrEv_GeneralInd_Connected_c:
358             App_UpdateStateLeds(gDeviceState_NwkConnected_c);
359             /* Set application CoAP destination to all nodes on connected network */
360             THR_GetIP6Addr(mThrInstanceId, gAllThreadNodes_c, &gCoapDestAddress, NULL);
361             APP_SetMode(mThrInstanceId, gDeviceMode_Application_c);
362             mFirstPushButtonPressed  = FALSE;
363             /* Synchronize server data */
364             THR_BrPrefixAttrSync(mThrInstanceId);
365             /* Enable LED for 80215.4 tx activity */
366             gEnable802154TxLed = TRUE;
367
368
369             /* Start counter timer (1 second) } */
370             if(mEquipo3TimerId == gTmrInvalidTimerID_c)
371             {
372                 mEquipo3TimerId = TMR_AllocateTimer();
373             }
374             if(mEquipo3TimerId != gTmrInvalidTimerID_c)
375             {
376                 gEquipo3Counter = APP_EQUIPO3_COUNTER_MIN_c;
377                 TMR_StartIntervalTimer(mEquipo3TimerId, APP_EQUIPO3_TIMER_PERIOD_c,
378                         APP_Equipo3TimerCallback, NULL);
379             }
380
381             /* Initialize accelerometer once network is up */
382             (void)NWKU_SendMsg((void*)APP_InitAccel, NULL, mpAppThreadMsgQueue);
383             /* Uncomment to register multicast address */
384             //IP_IF_AddMulticastGroup6(gIpIfSlp0_c, &mCastGroup);
385             break;
```

FUNCIONES NUEVAS

```
/*!*************************************************************************
*************************
```

```
\brief  Application.

*************************************************************************
*************************/
static void APP_CoapEquipo3Cb
(
    coapSessionStatus_t sessionStatus,
    uint8_t *pData,
    coapSession_t *pSession,
    uint32_t dataLen
)
{
      char addrStr[INET6_ADDRSTRLEN];
      char counterStr[8];
      uint32_t counterStrLen = 0;
      bool_t isConfirmable;

    /* Only process successfully received messages */
    if(sessionStatus != gCoapSuccess_c)
    {
        return;
    }

    /* ---- CON vs NON and print requester info ---- */
    isConfirmable = (pSession->msgType == gCoapConfirmable_c);

    ntop(AF_INET6, (ipAddr_t*)&pSession->remoteAddrStorage.ss_addr,
         addrStr, INET6_ADDRSTRLEN);

    shell_write("\r");
    if(isConfirmable)
    {
        shell_printf("CON instruction received from %s\n\r", addrStr);
    }
    else
    {
        shell_printf("NON instruction received from %s\n\r", addrStr);
    }
    shell_refresh();


    /* ---- Build counter string for GET responses ---- */
    if(pSession->code == gCoapGET_c)
    {
      /* Convert counter value to decimal ASCII string */
      uint16_t val = gEquipo3Counter;

      counterStr[0] = '\0';
      counterStrLen = 0;

      if(val >= 100)
      {
            counterStr[counterStrLen++] = (char)('0' + (val / 100));
            val %= 100;
            counterStr[counterStrLen++] = (char)('0' + (val / 10));
            counterStr[counterStrLen++] = (char)('0' + (val % 10));
      }
```

```
      else if(val >= 10)
      {
            counterStr[counterStrLen++] = (char)('0' + (val / 10));
            counterStr[counterStrLen++] = (char)('0' + (val % 10));
      }
      else
      {
            counterStr[counterStrLen++] = (char)('0' + val);
      }
      counterStr[counterStrLen] = '\0';
    }



    /* ---- Reply, ACK for CON, silence for NON ---- */
    if(isConfirmable)
    {
        if(pSession->code == gCoapGET_c)
        {
            /* Respond with counter value */
            COAP_Send(pSession, gCoapMsgTypeAckSuccessContent_c,
                      (uint8_t *)counterStr, counterStrLen);
        }
        else
        {

            COAP_Send(pSession, gCoapMsgTypeAckSuccessChanged_c, NULL,
0);
        }
    }

}


/*!*************************************************************************
*************************
\brief  Increments the counter from 1 to 200.
        Called from the task context via message queue.
***************************************************************************
*************************/
static void APP_Equipo3CounterTick
(
    uint8_t *pParam
)
{
    gEquipo3Counter++;

    if(gEquipo3Counter > APP_EQUIPO3_COUNTER_MAX_c)
    {
        gEquipo3Counter = APP_EQUIPO3_COUNTER_MIN_c;
    }
}



/*!*************************************************************************
*************************
```

```
\brief  Timer ISR-context callback that posts a counter-tick message to
the application queue.
        Fires every APP_EQUIPO3_TIMER_PERIOD_c milliseconds (1 second).
*************************************************************************
*************************/
static void APP_Equipo3TimerCallback
(
    void *param
)
{
    (void)NWKU_SendMsg(APP_Equipo3CounterTick, NULL,
mpAppThreadMsgQueue);
}

/*!*************************************************************************
*************************
\brief  Initializes the accelerometer over I2C.
        Scans the candidate I2C addresses, calls FXOS_Init(), and stores
the
        sensor range so raw readings can be used later.
*************************************************************************
*************************/
static void APP_InitAccel
(
    uint8_t *param
)
{
    i2c_master_config_t i2cConfig;
    uint32_t i2cSourceClock;
    uint8_t i;
    uint8_t regResult = 0;
    uint8_t arrayAddrSize = sizeof(mAccelAddress) /
sizeof(mAccelAddress[0]);
    bool_t foundDevice = FALSE;

    /* Get I2C source clock and wire up handles */
    i2cSourceClock = CLOCK_GetFreq(ACCEL_I2C_CLK_SRC);
    gFxosHandle.base      = BOARD_ACCEL_I2C_BASEADDR;
    gFxosHandle.i2cHandle = &gAccelMasterHandle;

    /* Initialize I2C master */
    I2C_MasterGetDefaultConfig(&i2cConfig);
    I2C_MasterInit(BOARD_ACCEL_I2C_BASEADDR, &i2cConfig, i2cSourceClock);
    I2C_MasterTransferCreateHandle(BOARD_ACCEL_I2C_BASEADDR,
&gAccelMasterHandle, NULL, NULL);

    /* Scan candidate addresses to find the sensor */
    for(i = 0; i < arrayAddrSize; i++)
    {
        gFxosHandle.xfer.slaveAddress = mAccelAddress[i];
        if(FXOS_ReadReg(&gFxosHandle, WHO_AM_I_REG, &regResult, 1) ==
kStatus_Success)
        {
            foundDevice = TRUE;
            break;
        }
    }
```

```
    if(!foundDevice)
    {
        shell_write("\r\nAccelerometer not found\r\n");
        shell_refresh();
        return;
    }

    /* Initialize FXOS sensor */
    if(FXOS_Init(&gFxosHandle) != kStatus_Success)
    {
        shell_write("\r\nAccelerometer init failed\r\n");
        shell_refresh();
    }
}


/*!*************************************************************************
*************************
\brief  Accelerometer Applicaation.
***************************************************************************
************************/

static void APP_CoapAccelCb
(
    coapSessionStatus_t sessionStatus,
    uint8_t *pData,
      coapSession_t *pSession,
      uint32_t dataLen
)
{
      char addrStr[INET6_ADDRSTRLEN];
      bool_t isConfirmable;
      char accelStr[48];
      uint32_t accelStrLen = 0;
      fxos_data_t sensorData;
      int16_t xData, yData, zData;

      /* Solo se procesan mensajes correctamente recibidos */
      if(sessionStatus != gCoapSuccess_c)
      {
            return;
      }

      /* CON vs NON and print requester info */
      isConfirmable = (pSession->msgType == gCoapConfirmable_c);

      ntop(AF_INET6, (ipAddr_t*)&pSession->remoteAddrStorage.ss_addr,
                  addrStr, INET6_ADDRSTRLEN);

      shell_write("\r");
      if(isConfirmable)
      {
            shell_printf("CON instruction received from %s\n\r",
addrStr);
      }
      else
```

```
        {
                shell_printf("NON instruction received from %s\n\r",
addrStr);
        }
        shell_refresh();

        /* Only reply GET requests */
        if(pSession->code != gCoapGET_c)
        {
                return;
        }

        /* Read raw X, Y, Z from accelerometer */
        if(FXOS_ReadSensorData(&gFxosHandle, &sensorData) ==
kStatus_Success)
        {
                /* Convert to 14-bit signed values */
                xData = (int16_t)((uint16_t)((uint16_t)sensorData.accelXMSB
<< 8) | sensorData.accelXLSB) / 4U;
                yData = (int16_t)((uint16_t)((uint16_t)sensorData.accelYMSB
<< 8) | sensorData.accelYLSB) / 4U;
                zData = (int16_t)((uint16_t)((uint16_t)sensorData.accelZMSB
<< 8) | sensorData.accelZLSB) / 4U;

                /* Format response string */
                accelStrLen = snprintf(accelStr, sizeof(accelStr), "x=%d y=%d
z=%d", xData, yData, zData);
        }
        else
        {
                accelStrLen = snprintf(accelStr, sizeof(accelStr), "accel
read error");
        }

    /* Send response with accel data for both CON and NON GET */
    if(isConfirmable)
    {
        COAP_Send(pSession, gCoapMsgTypeAckSuccessContent_c, (uint8_t
*)accelStr, accelStrLen);
    }
    else
    {
        COAP_Send(pSession, gCoapMsgTypeNonPost_c, (uint8_t *)accelStr,
accelStrLen);
    }

}
```

Modified code on Router_2.

This callback fires every 2 seconds, opens a CoAP session, and performs a CON GET toward the leader router's URI.

```
static void APP_Router2TimerCallback(void *param)
{
```

```
coapSession_t *pSession = COAP_OpenSession(mAppCoapInstId);

if (pSession)
{
pSession->code = gCoapGET_c;
pSession->msgType = gCoapConfirmable_c; // CON request
pSession->pCallback = APP_Router2CoapResponseCb;

/* Dirección destino del Leader o el Router1 */
FLib_MemCpy(&pSession->remoteAddrStorage.ss_addr, &gCoapDestAddress,
sizeof(ipAddr_t));

COAP_AddOptionToList(pSession, COAP_URI_PATH_OPTION,
(uint8_t *)APP_TEAM_URI_PATH,
SizeOfString(APP_TEAM_URI_PATH));

COAP_Send(pSession, gCoapMsgTypeConGet_c, NULL, 0);
}
}
```

The response callback is implemented to handle the incoming ACK with the counter value.

```
static void APP_Router2CoapResponseCb
(
coapSessionStatus_t sessionStatus,
uint8_t *pData,
coapSession_t *pSession,
uint32_t dataLen
)
{
char addrStr[INET6_ADDRSTRLEN];
char valueStr[12];

if (sessionStatus == gCoapSuccess_c && pData != NULL && dataLen > 0)
{
/* Obtener la IP */
ntop(AF_INET6, (ipAddr_t *)&pSession->remoteAddrStorage.ss_addr,
addrStr, INET6_ADDRSTRLEN);

/* Copiar valor recibido a string */
uint32_t len = (dataLen < sizeof(valueStr) - 1) ? dataLen :
(sizeof(valueStr) - 1);
FLib_MemCpy(valueStr, pData, len);
valueStr[len] = '\0';

/* Tipo de mensaje */
const char *msgType = (gCoapConfirmable_c == pSession->msgType) ? "CON" :
"NON";
```

```
mLastReceivedCount = (uint32_t)atoi(valueStr);


shell_printf("Counter = %s from %s type %s\r\n", valueStr, addrStr,
msgType);
shell_refresh();
}
}
```

In the `Stack_to_APP_Handler` function, inside the case for the event where the board registers that it has connected to the network, the 2-second timer initialization for Router 2 was added.

```
case gThrEv_GeneralInd_Connected_c:
App_UpdateStateLeds(gDeviceState_NwkConnected_c);
/* Set application CoAP destination to all nodes on connected network */
THR_GetIP6Addr(mThrInstanceId, gAllThreadNodes_c, &gCoapDestAddress,
NULL);
APP_SetMode(mThrInstanceId, gDeviceMode_Application_c);
mFirstPushButtonPressed = FALSE;
/* Synchronize server data */
THR_BrPrefixAttrSync(mThrInstanceId);
/* Enable LED for 80215.4 tx activity */
gEnable802154TxLed = TRUE;
/* Uncomment to register multicast address */
//IP_IF_AddMulticastGroup6(gIpIfSlp0_c, &mCastGroup);
//Iniciador del timer 2s
mRouter2TimerId = TMR_AllocateTimer();
if (mRouter2TimerId != gTmrInvalidTimerID_c)
{
TMR_StartIntervalTimer(mRouter2TimerId, 2000, APP_Router2TimerCallback,
NULL);
}
break;
```

Part 2

The defines for two new URIs were added to stop and resume the timer.

```
#define APP_STOPTIMER_URI_PATH                  "/stopTmr"
#define APP_RESTARTTIMER_URI_PATH               "/restartTmr"
```

The function prototypes were also added.

```
static void APP_CoapStopTmrCb(coapSessionStatus_t sessionStatus, uint8_t
*pData, coapSession_t *pSession, uint32_t dataLen);
static void APP_CoapRestartTmrCb(coapSessionStatus_t sessionStatus,
uint8_t *pData, coapSession_t *pSession, uint32_t dataLen);
```

Both callbacks were implemented. In APP_CoapStopTmrCb, the IP address of the router that sent the command is obtained, and whether it was CON or NON is determined. The timer is stopped if it is running, and the value at which it stopped is printed.

In APP_CoapRestartTmrCb, the same logic applies, except that instead of stopping the counter it resumes it from where it left off.

```
static void APP_CoapStopTmrCb
(
coapSessionStatus_t sessionStatus,
uint8_t *pData,
coapSession_t *pSession,
uint32_t dataLen
)
{
char addrStr[INET6_ADDRSTRLEN];

ntop(AF_INET6, (ipAddr_t *)&pSession->remoteAddrStorage.ss_addr, addrStr,
INET6_ADDRSTRLEN);

const char *msgType = (gCoapConfirmable_c == pSession->msgType) ? "CON" :
"NON";
TMR_StopTimer(mRouter2TimerId);
shell_printf("El tiempo se detuvo en %s tipo %s Contador= %lu\r\n",
addrStr,
msgType,
(unsigned long)mLastReceivedCount);
shell_refresh();
if (gCoapConfirmable_c == pSession->msgType)
{
COAP_Send(pSession, gCoapMsgTypeAckSuccessChanged_c, NULL, 0);
}
}
```

```
static void APP_CoapRestartTmrCb
(
coapSessionStatus_t sessionStatus,
uint8_t *pData,
coapSession_t *pSession,
uint32_t dataLen
)
{
char addrStr[INET6_ADDRSTRLEN];
ntop(AF_INET6, (ipAddr_t *)&pSession->remoteAddrStorage.ss_addr, addrStr,
INET6_ADDRSTRLEN);
const char *msgType = (gCoapConfirmable_c == pSession->msgType) ? "CON" :
"NON";

TMR_StartIntervalTimer(mRouter2TimerId, 2000, APP_Router2TimerCallback,
NULL);

shell_printf("El timer inicio en %s tipo %s Contador= %lu\r\n",
addrStr,
msgType,
(unsigned long)mLastReceivedCount);
shell_refresh();

if (gCoapConfirmable_c == pSession->msgType)
{
COAP_Send(pSession, gCoapMsgTypeAckSuccessChanged_c, NULL, 0);
}
}
```

It is worth noting that the URIs must be registered in APP_InitCoapDemo in order to be recognized.

```
static void APP_InitCoapDemo
(
void
)
{
coapRegCbParams_t cbParams[] = {{APP_CoapLedCb, (coapUriPath_t
*)&gAPP_LED_URI_PATH},
{APP_CoapTempCb, (coapUriPath_t *)&gAPP_TEMP_URI_PATH},

 {APP_CoapResource1Cb, (coapUriPath_t*)&gAPP_RESOURCE1_URI_PATH},

 {APP_CoapResource2Cb, (coapUriPath_t*)&gAPP_RESOURCE2_URI_PATH},

 //Parte 2   {APP_CoapStopTmrCb, (coapUriPath_t
*)&gAPP_STOPTIMER_URI_PATH},
 {APP_CoapRestartTmrCb, (coapUriPath_t *)&gAPP_RESTARTTIMER_URI_PATH},
```

```
#if LARGE_NETWORK
{APP_CoapResetToFactoryDefaultsCb, (coapUriPath_t
*)&gAPP_RESET_URI_PATH},
#endif
{APP_CoapSinkCb, (coapUriPath_t *)&gAPP_SINK_URI_PATH}};
/* Register Services in COAP */
sockaddrStorage_t coapParams = {0};
```

Personal conclusions.

Pablo:

One of the most valuable aspects was learning how CoAP operates as the application-layer protocol within a Thread network. Registering URIs, handling CON and NON message types, managing CoAP sessions, and correctly implementing callbacks required a level of attention to the system's architecture that goes beyond simply writing code. A small misunderstanding of how the timer worked or where a variable was scoped was enough to produce behavior that was difficult to debug, as the system gave no obvious error at runtime.

The practice also reinforced the importance of understanding the boundaries between projects. Since Router 1 and Router 2 are compiled and flashed as separate binaries running on independent devices, assumptions about shared variables or shared state are simply not valid. This is a fundamental concept in distributed embedded systems that is easy to overlook when working primarily in single-device environments.

Nahum:

The development of this practice represented both a technical and an organizational challenge. Due to personal circumstances of both team members, it was not possible to meet as frequently as desired, which required distributing the work more independently and coordinating primarily remotely. Despite this, we managed to meet the vast majority of the stated objectives.

From a technical standpoint, the practice allowed us to understand in an applied way how a Thread network works and the role that the CoAP protocol plays in communication between nodes. Implementing the `/Equipo3` and `/accel` endpoints required understanding the SDK's base code and also how to extend it correctly without breaking the existing flow.

Working with the accelerometer was particularly enriching, as it required configuring the I2C bus, meaning the work was not limited to the network layer alone.

In conclusion, although the circumstances were not ideal, the practice provided solid knowledge about Thread networks and CoAP communication — knowledge considered relevant for adequate training in this field.